CS 0007: Introduction to Java Lecture 8

Nathan Ong
University of Pittsburgh
September 27, 2016

Announcement

- Your midterm exam is scheduled for Tuesday, October 11, during the normal class period.
- Anything from the lecture, homework, quiz, labs, is fair game.
- You should also be comfortable with analyzing previously unseen code in conjunction with using the API (as seen in several homeworks).

Function Components

Function Header

1. Visibility type (`public`/`protected`/`private`)
2. `static` (For now, required)
3. Return Type
4. functionName
5. Parentheses “()”
   - Parameters
   a) Type1 parameterName1
   b) Type2 parameterName2
   c) …

Function Body

6. Curly Brackets/Braces “{}”
   - `return` a value

Function Overloading

- Sometimes, the function name that was already used fits perfectly with a function you need to create
- They have similar functions, but operate on different parameters
- Turns out, you can use the same name, in what is called function overloading

Function Overloading Rules

- Functions with the same name MUST have different typed parameters, or a different number of parameters.
- We have seen this before!!! (**`System.out.println()`**)

Examples

```
public static void britishGreeting
    (String fullName)

public static void britishGreeting
    (String yourName, String childName)

public static int britishGreeting
    (int numOfPeople)

public static ch       ishGreeting
    (String firstNameOnly)
```



Special Functions

- Constructors: Functions that create new objects, preceded by the keyword `new` before calling it.
- main: The first function that is always run by any Java program. It always has a `String[]` as the only parameter.
- Functions common to all objects: we may return to this later.

And More Booleans and Scope

IF STATEMENTS

If Statements

- English usage?
- “If it will rain today, then I will bring my umbrella.”
- Rain today would indicate that I have brought my umbrella
- Cause and effect relationship: when a condition is met, a behavior is elicited.

Skeleton Statement

```
if(<boolean condition>)
{
    …
}
…
```

Boolean Condition

- Must evaluate to true in order for the statements within the `if`-block to execute.
- Evaluating to false will skip the block.

Java Example

```
  /*  checkWeather returns true if
  it
   *  will rain today.*/
boolean rainToday = checkWeather();
if(rainToday)
{
      bringUmbrella();
}
goToSchool();
```



Remember this?
"has a Facebook: true“
How do we fix this?

Code Reminder

```
public class Name
{
  public static void main(String[] args)
  {
  String firstName = "Nathan";
  char midInitial = 'R';
  String lastName = "Ong";
  int age = 19;
  boolean hasFacebook = true;
  System.out.println(firstName + " " + midInitial +    " " + lastName + ",
    age: "    + age +    ", has a Facebook: " + hasFacebook);
  }//end method main
}//End class Name
```

A More Useful Output

- if hasFacebook is true, then print what?
- "has a Facebook"
- Otherwise, if hasFacebook is false, then print what?
- "does not have a Facebook"

if()

- if hasFacebook is true, then print "has a Facebook"

```
if(hasFacebook is true)
{
        System.out.println("has a Facebook");
}//end block if(hasFacebook is true)
```

hasFacebook is true

- How do we model this?
- `hasFacebook = true; //?`
- But that let us do name-changing!
- `name = "Brandon Ong";`

Making a new thing:
Type name = value;

Changing the old thing:
name = newValue;

hasFacebook is true

- How do we model this?
- `hasFacebook = true //?`
- But that let us do name-changing!
- `name = "Brandon Ong";`
- So no, we can't do this to check for equality
- We instead use double equals (==)

== vs =

- = is for assignment
  - `String name = "Nathan Ong";`
  - name has value?
  - `int age = 19;`
  - age has value?
- == is for comparison (equal or not equal)
  - `age == 19;`
  - `name == "Nathan Ong";`
- What values do we get for the last two statements?
- What type are they?

WARNING

- Be careful when comparing non-primitive values!!!
- **`name == "Nathan Ong"`** could actually give us **`false`**!
- Instead you will need to use **`equals()`**
- **`name.equals("Nathan Ong");`**

if()

- if hasFacebook is true, then print "has a Facebook"

```
if(hasFacebook == true)
{
  System.out.println("has a
  Facebook");
}//end block if(hasFacebook == true)
```

- But how do I represent when hasFacebook is false?

`else if()`

- If an `if` condition fails (the boolean condition evaluates to false), it will check the condition for the subsequent `else if`.
- Can also be placed directly after another `else if` block (i.e. you can have several).

else if()

- Otherwise, if hasFacebook is false, then print "does not have a Facebook"

```
else if(hasFacebook == false)
{
  System.out.println("does not
      have a Facebook");
}//end block else if(hasFacebook
 // == false)
```

else if()

```
if(hasFacebook == true)
{
  System.out.println("has a Facebook");
}//end block if(hasFacebook == true)
else if(hasFacebook == false)
{
  System.out.println("does not have a
   Facebook");
}//end block else if(hasFacebook == false)
```

Seems kinda wordy...

`hasFacebook == true`

- What does that give us?
- What type?
- `hasFacebook == true` when `hasFacebook = true` gives us?
- `hasFacebook == true` when `hasFacebook = false` gives us?
- `hasFacebook == true` → `hasFacebook`
- *Gasp*

if()

```
if(hasFacebook)
{
  System.out.println("has a Facebook");
}//end block if(hasFacebook)
else if(hasFacebook == false)
{
  System.out.println("does not have a
   Facebook");
}//end block else if(hasFacebook == false)
```

Still kinda wordy...

`hasFacebook == false`

- We want to use the same trick we did before, except this time with `false` instead of `true`.
- Remember the `boolean` operators?

if()

```
if(hasFacebook)
{
  System.out.println("has a Facebook");
}//end block if(hasFacebook)
else if(!hasFacebook)
{
  System.out.println("does not have a
   Facebook");
}//end block else if(!hasFacebook)
```

Still kinda wordy...